
Refatoração

Manutenção de Software

- Preventiva: bugs evidentes
- Corretiva: bugs reportados por usuários
- Adaptativa: customizações, novas versões sistemas associados
- Evolutiva: novas funcionalidades
- Refatoração: melhorias no código ou design

Leis de Lehman (resumo)

- Sistemas devem ser sempre mantidos para serem úteis
- Mas manutenções aumentam a complexidade e deterioram a qualidade do código
- A não ser que a **refatoração** seja realizada para evitar isso:
 - Refatoração = transformações de código que melhoram a manutenibilidade de um sistema mas sem afetar o seu funcionamento externo

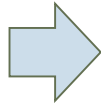
Catálogo de Refatoração

- Extração de Métodos
- *Inline* de Métodos
- Movimentação de Métodos
- Extração de Classes
- Renomeação
- etc

Extração de Métodos

Extração de Métodos

```
void f () {  
    ... // A  
    ... // B  
    ... // C  
}
```



```
void g() { // método extraído  
    ... // B  
}  
  
void f () {  
    ... // A  
    g(); // chama método extraído  
    ... // C  
}
```

Antes

```
void onCreate(SQLiteDatabase database) { // antes da extração
    // cria tabela 1
    database.execSQL("CREATE TABLE " +
        CELL_SIGNAL_TABLE + " (" + COLUMN_ID +
        " INTEGER PRIMARY KEY AUTOINCREMENT, " + ...
    database.execSQL("CREATE INDEX cellID_index ON " + ...);
    database.execSQL("CREATE INDEX cellID_timestamp ON " + ...);

    // cria tabela 2
    String SMS_DATABASE_CREATE = "CREATE TABLE " +
        SILENT_SMS_TABLE + " (" + COLUMN_ID +
        " INTEGER PRIMARY KEY AUTOINCREMENT, " + ...
    database.execSQL(SMS_DATABASE_CREATE);
    String ZeroSMS = "INSERT INTO " + SILENT_SMS_TABLE +
        " (Address,Display,Class,ServiceCtr,Message) " +
        "VALUES ('"+ ...
    database.execSQL(ZeroSMS);

    // cria tabela 3
    String LOC_DATABASE_CREATE = "CREATE TABLE " +
        LOCATION_TABLE + " (" + COLUMN_ID +
        " INTEGER PRIMARY KEY AUTOINCREMENT, " + ...
    database.execSQL(LOC_DATABASE_CREATE);
    // mais 200 linhas, criando outras tabelas
}
```

```
public void onCreate(SQLiteDatabase database) {  
    createCellSignalTable(database);  
    createSilentSmsTable(database);  
    createLocationTable(database);  
    createCellTable(database);  
    createOpenCellIDTable(database);  
    createDefaultMCCTable(database);  
    createEventLogTable(database);  
}
```

Depois

Motivações para Extração de Métodos	Ocorrências
Extração de um método para ser reusado em outras partes do código	43
Introduzir uma assinatura alternativa para um método existente	25
Dividir um método em partes menores para melhorar o seu entendimento	21
Remover duplicação de código	15
Extração de um método para facilitar a implementação de uma nova funcionalidade ou a correção de um bug	14
Extração de um método que terá um nome melhor ou menos parâmetros. O método original é depreciado e delega as suas chamadas para o método extraído	6
Permitir o teste do método extraído, isto é, melhorar testabilidade	6
Permitir que subclasses sobrescrevam o método que foi extraído	4
Permitir a implementação de um método recursivo	2
Introduzir um método fábrica, isto é, as chamadas de new são movidas para o novo método	1
Implementar operações assíncronas, isto é, o método é extraído para ser executado em uma thread independente	1

Danilo Silva, Nikolaos Tsantalis, Marco Tulio Valente.
Why We Refactor? Confessions of GitHub Contributors.
FSE 2016.

Inline de Métodos (contrário de extração)

```
private void writeContentToFile(final byte[] revision) {  
    getVirtualFile().setBinaryContent(revision);  
}
```

```
private void write(byte[] revision) {  
    VirtualFile virtualFile = getVirtualFile();  
    ...  
    if (document == null) {  
        writeContentToFile(revision); // única chamada  
    }  
    ...  
}
```

Antes

```
private void writeContentToFile(final byte[] revision) {  
    getVirtualFile().setBinaryContent(revision);  
}  
  
private void write(byte[] revision) {  
    VirtualFile virtualFile = getVirtualFile();  
    ...  
    if (document == null) {  
        writeContentToFile(revision); // única chamada  
    }  
    ...  
}
```

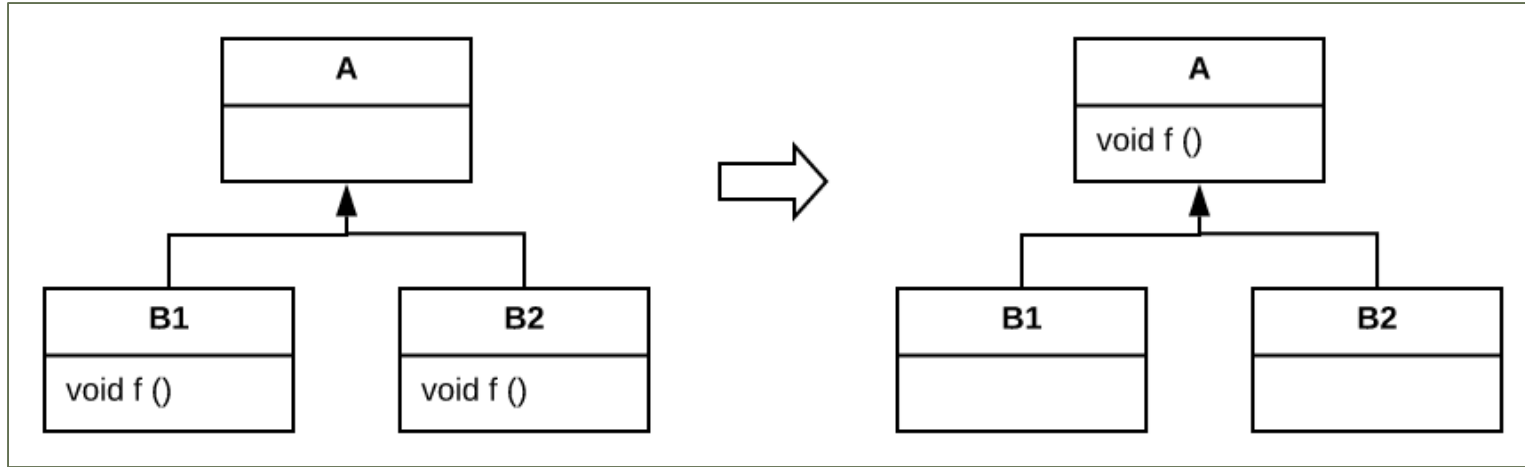


```
private void write(byte[] revision) {  
    VirtualFile virtualFile = getVirtualFile();  
    ...  
    if (document == null) {  
        virtualFile.setBinaryContent(revision); // após inline  
    }  
    ...  
}
```

Depois

Movimentação de Métodos

Pull Up Method

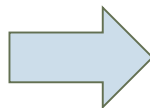


```
graph BT; B1 --> A; B2 --> A; A --> B1; A --> B2; A --- M1[void f()]; B1 --- M2[ ]; B2 --- M3[ ]
```

The diagram shows a transformation of a class hierarchy. On the left, class A is the base of B1 and B2. A has a method 'void f()'. B1 and B2 are subclasses. An arrow points to the right, showing the result: class A's method 'void f()' has been removed, while B1 and B2 remain unchanged.

Extração de Classes


```
class Pessoa {  
    String codigoArea;  
    String fone;  
    String codigoAreaCelular;  
    String celular;  
    ...  
}
```



```
class Fone { // classe extraída  
    String codigoArea;  
    String numero;  
}  
  
class Pessoa {  
    Fone foneFixo;  
    Fone foneCelular  
    ...  
}
```

Renomeação

(variáveis, parâmetros, métodos, classes, exceções, etc)

A solid orange horizontal bar spanning the width of the slide at the bottom.

Renomação é a refatoração mais popular

Refactoring Tool	Toolsmiths				Mylyn				Users				Everyone	
	Uses	Use %	Batched	Batched %	Uses	Use %	Batched	Batched %	Uses	Use %	Batched	Batched %	Uses	Use %
Rename	670	28.7%	283	42.2%	2706	53.6%	1146	42.4%	1862	61.5%	1009	54.2%	49672	71.8%
Extract Local Variable	568	24.4%	127	22.4%	260	5.2%	57	21.9%	322	10.6%	106	32.9%	4917	7.1%
Inline	349	15.0%	132	37.8%	110	2.2%	52	47.3%	137	4.5%	52	38.0%	1426	2.1%
Extract Method	280	12.0%	28	10.0%	316	6.3%	27	8.5%	259	8.6%	57	22.0%	3345	4.8%

Murphy-Hill, et al. How We Refactor, and How We Know It. IEEE TSE 2012.

Prática de Refatoração

Quando refatorar?

1. Refatoração oportunista:

- Realizada no meio de uma outra tarefa de programação
- Tipo de refatoração mais comum

2. Refatoração planejada:

- Correção de um problema mais complexo
- Iteração apenas para realização de refatorações

Refatoração Automatizada

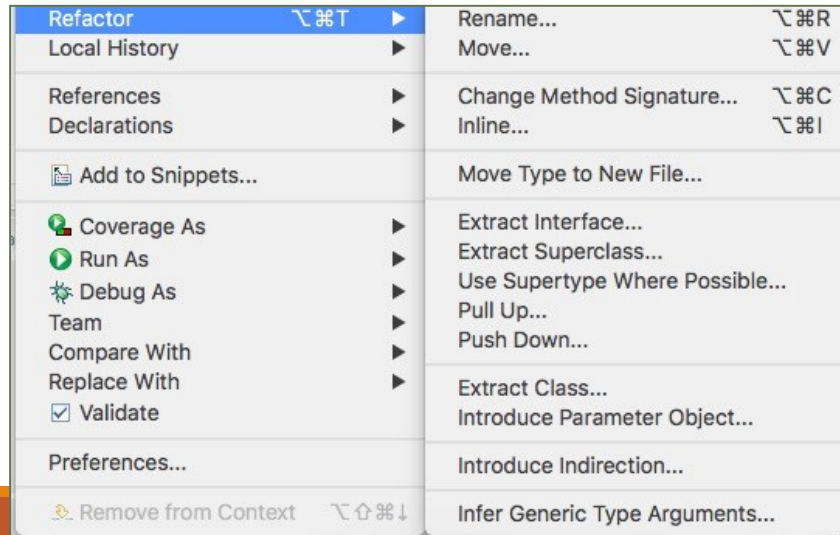
(realizados com auxílio de uma IDE)

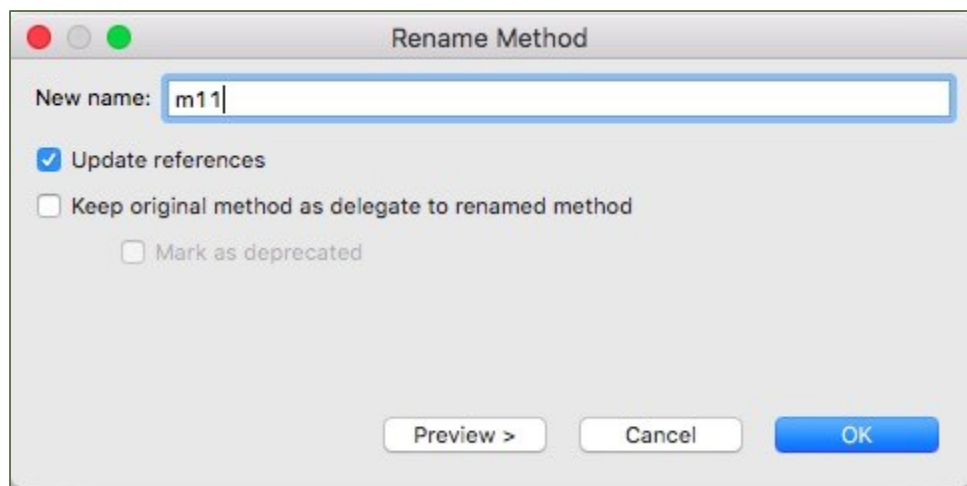
```

class A {
    void m1() {}
    void m2() {}
    void m3() { m1(); }
}

class B {
    void m4() { new A().m1(); }
}

```





```
class A {  
    void m11() {}  
    void m2() {}  
    void m3() { m11(); }  
}  
  
class B {  
    void m4() { new A().m11(); }  
}
```

Code Smells

Code (ou Bad) Smells

- Indicadores de código de baixa qualidade
- Difícil de manter, entender, modificar ou testar
- Portanto, candidato a refatoração

Catálogo de Code Smells

- Código Duplicado
- *Feature Envy*
- Métodos Longos
- Classes Grandes
- Métodos com Muitos Parâmetros
- Variáveis Globais
- Obsessão por Tipos Primitivos
- Documentação

Código Duplicado

Código Duplicado

- Dificulta a manutenção
- Portanto, código candidato a refatoração

RANKING OF MOST POPULAR CODE SMELLS/ANTI-PATTERNS

Smell/Anti-Pattern	Points
1. Duplicated code	19.53
2. Long method	9.78
3. Accidental complexity	8.32
4. Large class	7.09
5. Excessive use of literals	3.04
6. Suboptimal information hiding	2.70
7. Lazy class	2.33
8. Feature Envy	2.33
9. Long parameter list	2.31
10. Dead code	2.25
11. Bad (or lack of good) comments	1.50
12. Use deprecated components	1.50
13. Single Responsibility	1.20
14. Complex conditionals	1.12
15. Bad naming	1.12

Aiko Yamashita, Leon Moonen. Do developers care about code smells? An exploratory survey. WCRE 2013.

Código Duplicado $\Rightarrow$ Clones

Clone Tipo 1 (comentários e espaços)

```
int fatorial(int n) {  
    fat = 1;  
    for (i = 1; i <= n; i++)  
        fat = fat * i;  
    return fat;  
}
```

Código Original

```
int fatorial(int n) {  
    fat=1;  
    for (i=1; i<=n; i++)  
        fat=fat*i;  
    return fat; // retorna fatorial  
}
```


Clone Tipo 2 (tipo 1 + nomes diferentes)

```
int fatorial(int n) {  
    fat = 1;  
    for (i = 1; i <= n; i++)  
        fat = fat * i;  
    return fat;  
}
```

Código Original

```
int fatorial(int n) {  
    f = 1;  
    for (j = 1; j <= n; j++)  
        f = f * j;  
    return f;  
}
```

Clone Tipo 3 (tipo 2 + mudanças em comandos)

```
int fatorial(int n) {  
    fat = 1;  
    for (i = 1; i <= n; i++)  
        fat = fat * i;  
    return fat;  
}
```

Código Original

```
int fatorial(int n) {  
    fat = 1;  
    for (j = 1; j <= n; j++)  
        fat = fat * j;  
    System.out.println(fat); // novo comando  
    return fat;  
}
```

Clone Tipo 4 (algoritmos diferentes, mas equivalentes)

```
int fatorial(int n) {  
    fat = 1;  
    for (i = 1; i <= n; i++)  
        fat = fat * i;  
    return fat;  
}
```

Código Original

```
int fatorial(int n) {  
    if (n == 0)  
        return 1;  
    else return n*fatorial(n-1);  
}
```

Feature Envy

Feature Envy

- Método que "inveja" dados e métodos de outra classe
- Usa mais métodos e dados dessa outra classe
- Logo, é um candidato a ser movido para ela

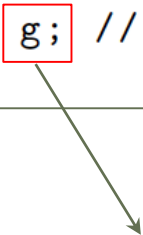
```
public class DrawingEditorProxy
    extends AbstractBean implements DrawingEditor {
    ...
    void fireAreaInvalidated2 (AbstractTool abt , Double r ){
        Point p1 = abt.getView().drawingToView (...);
        Point p2 = abt.getView().drawingToView (...);
        Rectangle r=new Rectangle(p1.x,p1.y,p2.x-p1.x p2.y-p1.y);
        abt.fireAreaInvalidated (r);
    }
    ...
}
```

Variáveis Globais

Variáveis Globais

- Acoplamento ruim
- Dificulta o entendimento de um método

```
void f(...) {  
    // computa um determinado valor x  
    return x + g; // onde g é uma variável global  
}
```



Para entender o que `f` retorna, precisamos conhecer o valor de `g`
Esse valor pode variar entre chamadas de `f`, etc

Obsessão por Tipos Primitivos

Obsessão por Tipos Primitivos

-
- CEP, Moeda, Data, Hora, Cor, Email, etc **não** devem ser de tipos primitivos
 - Mas sim de um tipo próprio, com alguns métodos
 - Por exemplo, métodos para validação de valores

Comentários

"não comente código ruim, reescreva-o"

-- B. Kernighan & P. J. Plauger

A solid orange horizontal bar at the bottom of the slide.

Comentários "Ruído" (nada acrescentam ...)

```
/** The day of the month. */  
private int dayOfMonth;
```

```
// this function sends an email  
void sendEmail() {  
    ...  
}  
  
// this class holds data for an employee  
public class Employee {  
    ...  
}
```

Mais um exemplo de comentários "ruído"

```
// Add a horizontal scroll bar  
hScrollBar = new JScrollBar(JScrollBar.HORIZONTAL);  
add(hScrollBar, BorderLayout.SOUTH);
```

```
// Add a vertical scroll bar  
vScrollBar = new JScrollBar(JScrollBar.VERTICAL);  
add(vScrollBar, BorderLayout.EAST);
```

```
// Initialize the caret-position related values  
caretX    = 0;  
caretY    = 0;  
caretMemX = null;
```

Comentários apenas repetem o que já está bem claro no código

Quando é útil comentar?

- Código complexo por natureza

```
// format matched kk:mm:ss EEE, MMM dd, yy  
Pattern timePattern = Pattern.compile("\\d*:\\d*:\\d* \\w*, \\w*, \\d*, \\d*");
```

Quando é útil comentar?

- Documentação de APIs

```
/**
 * Registers the text to display in a tool tip. The text
 * displays when the cursor lingers over the component.
 *
 * @param text  the string to display.  If the text is null,
 *              the tool tip is turned off for this component.
 */
public void setToolTipText(String text) {
```

Débito Técnico

Débito Técnico



- Metáfora para explicar a importância de boas práticas e princípios de Engenharia de Software
- Proposto por Ward Cunningham (1992)
- Soluções não-ótimas que dificultam a manutenção e evolução de um sistema



Exemplos de Débito Técnico

- Ausência de testes
- Ausência de builds automatizadas
- Problemas arquiteturais
- Alto acoplamento e baixa coesão
- Code smells
- Ausência completa de documentação
- Código que não segue layout pré-definido

Referências

- [1] Chen Q., Grundy J., Hosking J.: SUMLOW: early design-stage sketching of UML diagrams on an E-whiteboard. Software Practice and Experience, 2008.
- [2] Larman, C. Applying UML and Patterns. Pearson, 2004.
- [3] Valente, M. T. Engenharia de Software Moderna. Editora Independente, 2020
- [4] Wazlawick, R. S. Análise e Projeto de Sistemas Orientados a Objetos. Elsevier, 2004.